

Wafer Defect Classification Final Milestone Presentation Script

Najani Johnson and Meghan Herbert

Slide 1: Title

Hello everyone. Today we are presenting our final milestone for our wafer defect classification project.

Our project focuses on using machine learning to classify semiconductor wafer defect patterns from the WM811K dataset. Each wafer map shows which die locations passed or failed testing, and the overall pattern can give engineers useful information about possible manufacturing issues.

For this milestone, we focused on improving the reliability of our model comparisons, fixing issues from the previous milestone, and testing different ways to handle class imbalance.

Slide 2: Previous Milestone Recap

In our previous milestone, we trained and compared a Random Forest model and CNN models for wafer defect classification.

The graphs on this slide summarize our CNN training behavior and accuracy comparison, while the confusion matrix shows where the models were making correct and incorrect predictions.

At that stage, the models looked strong overall, but the results also showed that accuracy alone did not tell the full story. Because the dataset contains many more **none** wafers than rare defect classes, a model could appear accurate while still struggling on the smaller defect categories.

Slide 3: Previous Milestone Key Limitations

After reviewing the previous milestone, we identified three main limitations.

First, our validation setup was inconsistent. Some models were being compared using different splits, which made the results less fair.

Second, the CNN architecture had limitations. It worked reasonably well, but it still needed better tuning and training controls.

Third, class imbalance was affecting our results. The old model performed well on **none** wafers because that class dominates the dataset, but it was less reliable on rare defect types.

We also tested the model on unseen examples, and it correctly classified 3 out of 6. That helped us see that the high accuracy did not fully capture real generalization across all classes.

Slide 4: Fixing Random Forest Input Formatting

One issue we fixed was the Random Forest input formatting.

Random Forest does not take a two-dimensional wafer image in the same way a CNN does, so we flattened the full training wafer maps into one-dimensional feature vectors.

We also made sure the flattened training data matched the correct label set, `y_train_full`. This avoided accidentally using labels from the smaller CNN split.

This fix was important because the Random Forest is our baseline model. If the baseline is not using correctly matched inputs and labels, then the whole comparison becomes less trustworthy.

Slide 5: Fixing Inconsistent Validation Setup

Next, we fixed the validation setup.

Before, each model could end up using a slightly different split of the data. That made it harder to know whether performance differences came from the model itself or from the data split.

After the fix, we added stratified train-test splitting so each split preserves the class distribution as much as possible. We also created one consistent validation set and stopped re-splitting during model training.

For the CNN, we added training controls like `ReduceLROnPlateau` and early stopping. These helped the model adjust the learning rate when validation performance stopped improving and reduced the chance of overtraining.

Overall, this made the old CNN versus improved CNN comparison much fairer.

Slide 6: Addressing Class Imbalance, Part 1

The next problem was class imbalance, so we compared two strategies: class weights and oversampling.

Class weights change the loss function during training. Mistakes on rare classes count more heavily, so the model is encouraged to pay more attention to those classes.

Oversampling works differently. Instead of changing the loss function, it increases the number of minority-class examples by repeating existing rare-class wafer maps.

Both methods are trying to reduce majority-class bias, but they have different tradeoffs. Class weighting keeps the dataset size more natural, while oversampling gives the model more minority-class examples but can also repeat the same patterns.

Slide 7: Addressing Class Imbalance, Part 2

This slide shows the results from oversampling compared with the results from adding class weights.

The main takeaway is that oversampling and class weighting did not behave the same. Oversampling gave the model more exposure to rare classes, but because it repeats existing wafers, it can also make the model learn those repeated examples too closely.

Class weights gave a cleaner improvement for our CNN because they directly changed how the model treated errors without duplicating the training images.

Based on these results, the improved CNN with class weights was the better imbalance-handling approach for this milestone.

Slide 8: Results of All Fixes

After applying the fixes, we compared the before and after results.

The biggest improvement was not just a single accuracy number. The more important improvement was that the comparison became more reliable.

The Random Forest baseline remained strong, which tells us that classical machine learning can still work very well when the wafer maps are resized and flattened properly.

The improved CNN became a fairer model to compare because it used the same validation setup and had better training controls. However, class imbalance still affected the rare defect classes, so the final results show progress but not a completely solved problem.

Slide 9: Demo

For the demo, we will show the project output and walk through how the models classify wafer maps.

First, we will point out the dataset preprocessing step, where wafer maps are cleaned, resized, and encoded into labels.

Then we will show the model comparison results, including the Random Forest baseline, the old CNN, and the improved CNN.

Finally, we will look at example predictions. This is useful because it connects the metrics back to actual wafer maps and shows where the model succeeds or struggles on specific defect patterns.

Slide 10: Key Takeaways and Next Steps

To summarize, our main takeaways are:

- Cleaner splits made the model comparisons more reliable.

- The Random Forest remained the strongest baseline.
- The improved CNN performed better with class weights than with oversampling.
- Class imbalance still affects the rare wafer defect classes.

For next steps, we would tune the CNN architecture and learning rate further, track per-class recall and F1 score more closely, and test stronger imbalance methods beyond simple oversampling.

We would also add more validation using truly unseen wafer-map examples, because that gives a better sense of how useful the model would be outside the training dataset.

Overall, this milestone helped us move from a basic model comparison toward a cleaner and more realistic evaluation of wafer defect classification.